

Cracking a Bank's Disputes & Fraud-Triage Problem with Agents

One enterprise problem, end to end: from a fresh AWS account to a governed, observable agentic system on Amazon Bedrock AgentCore — with real code, IAM, deploy steps and a go-live runbook.

Prepared for Kaniz

Domain: financial services / retail banking.

Builds on Guide 1 (Foundations). Pairs with Guide 3 (Enterprise MCP).

How to read this guide

Each phase has the concept, the AWS/AgentCore steps, and the code. Blue = key idea, green = best practice, orange = pitfall, dark = runnable code / CLI. Code is illustrative-but-real; adapt ARNs, IDs and schemas to your estate.

Contents

- 1 The business problem & success metrics
- 2 Solution architecture (the agentic design)
- 3 Phase 0 — AWS account & landing zone from zero
- 4 Phase 1 — Data & model foundations
- 5 Phase 2 — Build the agent (Strands)
- 6 Phase 3 — Expose core-banking APIs as tools (Gateway)
- 7 Phase 4 — Memory (conversation + customer context)
- 8 Phase 5 — Identity & authentication
- 9 Phase 6 — Guardrails & Policy (the safety net)
- 10 Phase 7 — Deploy to AgentCore Runtime
- 11 Phase 8 — Observability & Evaluations
- 12 Security & compliance deep dive
- 13 Cost model & estimate
- 14 End-to-end data flow (one dispute, start to finish)
- 15 Go-live runbook, failure modes & rollback

1 · The business problem & success metrics

The bank. A retail bank processes a high volume of **customer disputes** ("I didn't make this charge", "charged twice", "goods never arrived") and **fraud alerts** from its monitoring system. Today, human analysts manually gather transaction history, check fraud signals, apply policy, and decide: open a dispute case, temporarily block a card, issue a provisional credit, or clear the alert. It's slow, inconsistent, and expensive, and SLAs (e.g. regulatory dispute-resolution timeframes) are under pressure.

What we're building

An **agentic triage assistant** that, for each dispute or fraud alert, autonomously gathers the context, applies bank policy, performs **low-risk actions** itself (open a case, request documents, clear a clearly-false alert), and **escalates high-risk or money-moving actions to a human** with a structured recommendation. It serves both an internal analyst console and a customer-facing chat for status.

Success metrics (define these first)

Metric	Baseline → target	Why it matters
Avg handling time / case	12 min → < 4 min	Core efficiency case
% auto-resolved (low-risk)	0% → 40-55%	Analyst capacity freed
Policy-consistency	variable → auditable	Regulatory + fairness
SLA breach rate	X% → near 0	Avoid regulatory penalties
Cost / case	\$ → lower + measured	Justifies the build

Enterprise framing rule

Start from metrics and the riskiest decision, not the tech. The agent's job is to compress analyst time on the *easy* majority while routing the *hard, costly* minority to humans with better context. Money-moving actions stay gated. That single principle shapes every design choice below.

2 · Solution architecture (the agentic design)

The data-flow skeleton, in text (read top to bottom):

```

Analyst console / customer chat
  | (OAuth bearer token, session id)
  v
AgentCore Runtime --- Identity (inbound auth) validates caller
  [ Triage Agent: Strands + Claude on Bedrock ]
    | \__ Memory (conversation + customer/case context)
    | \__ Guardrails (PII redaction, denied topics) on model I/O
    |
    v MCP tool calls
AgentCore Gateway --- Policy (Cedar) checks each tool call
  |-- get_transactions (OpenAPI -> core banking, egress auth)
  |-- get_fraud_signals (Lambda target)
  |-- open_dispute_case (Lambda; idempotent; WRITE)
  |-- block_card (Lambda; HIGH RISK -> human approval)
  |
  v
Core banking / fraud / case-management systems
  |
  v
Everything emits OTEL traces -> AgentCore Observability + CloudWatch

```

Component-to-service mapping:

Layer	Service	Responsibility
Brain	Bedrock (Claude)	Reasoning + tool selection
Agent host	AgentCore Runtime	Serverless, session-isolated execution
Tools	AgentCore Gateway + Lambda/OpenAPI	Calls to enterprise systems as MCP tools
State	AgentCore Memory + DynamoDB	Conversation + durable case context
Auth	AgentCore Identity (+ Cognito/Okta)	Who can call; how agent calls backends
Safety	Guardrails + Policy (Cedar)	Content safety + deterministic action limits
Ops	Observability + Evaluations	Traces, metrics, quality scoring

3 · Phase 0 — AWS account & landing zone from zero

Before a line of agent code, stand up a safe, auditable footprint. (Foundations in Guide 1, Part C.)

- 1 Create an **Organization** with separate accounts for **dev / staging / prod** (blast-radius isolation; prod data never in dev).
- 2 Enable **IAM Identity Center** for human access; create permission sets (Developer, ReadOnly, SecurityAudit). No daily root use; root has MFA.
- 3 Turn on **CloudTrail** (org-wide trail to a locked logging account) and **AWS Config** for continuous audit.

- 4 Set **budgets + alarms** per account; mandate tags (*project=disputes-agent, env, owner, data-class*).
- 5 Choose a Region with Bedrock + AgentCore; enable **Bedrock model access** for Claude in that Region.
- 6 Define a **KMS** customer-managed key for the project; encrypt S3, DynamoDB, and logs with it.
- 7 Pick your IaC tool (CDK or Terraform) and create a repo; everything from here is code-reviewed and deployed via pipeline.

Compliance pitfall

In banking, data residency and segregation are not optional. Keep customer/transaction data in approved Regions, in private subnets, KMS-encrypted, with access logged. Decide your data-classification scheme now — it drives IAM, networking and which fields the agent may ever see.

4 · Phase 1 — Data & model foundations

- **DynamoDB** table *CaseStore* (PK=case_id) for durable case records + an idempotency table for write tools.
- **S3** bucket (KMS-encrypted, block-public-access on) for dispute documents and the OpenAPI specs Gateway will use.
- **Bedrock Knowledge Base** over the bank's **dispute & fraud policy** documents in S3, so the agent grounds decisions on current policy rather than memorised guesses.
- **VPC** with private subnets + **VPC endpoints** for Bedrock/S3/DynamoDB so traffic to core systems never touches the public internet.

Why a Knowledge Base here

Dispute rules change (thresholds, provisional-credit windows, regulatory timeframes). Putting them in a Knowledge Base means you update a document, not a prompt or code — and the agent cites the policy it applied, which auditors love.

5 · Phase 2 — Build the agent (Strands)

We use **Strands Agents** for the fastest AWS-native path; the same shape works with LangGraph/CrewAI. Start locally, tools as plain functions, then move them behind Gateway in Phase 3.

System prompt = the operating policy

```
SYSTEM = '''You are a bank disputes & fraud triage assistant.
Goals: gather context, apply policy from the knowledge base, and either
resolve LOW-RISK cases or escalate with a clear recommendation.
Rules you must never break:
- Never move money or block a card without human approval.
- Always cite the policy section you applied.
- If identity is unverified, do not disclose account details.
- Prefer asking one clarifying question over guessing.'''
```

The agent + a couple of tools

```
from strands import Agent, tool
from strands.models import BedrockModel

@tool
def get_transactions(account_id: str, days: int = 30) -> list[dict]:
    """Return the customer's recent transactions for triage."""
    return core_bank.transactions(account_id, days) # real API in Phase 3

@tool
def get_fraud_signals(transaction_id: str) -> dict:
    """Fraud-model score and signals for a transaction."""
    return fraud.signals(transaction_id)

@tool
def open_dispute_case(account_id: str, transaction_id: str, reason: str,
                      idempotency_key: str) -> dict:
    """Open a dispute case (LOW-RISK write). Idempotent."""
    return cases.open(account_id, transaction_id, reason, idempotency_key)

model = BedrockModel(model_id='anthropic.claude-3-5-sonnet-20240620-v1:0',
                      temperature=0.2, region_name='us-east-1')

agent = Agent(model=model, system_prompt=SYSTEM,
              tools=[get_transactions, get_fraud_signals, open_dispute_case])

result = agent('Customer 4471 disputes a $240 charge they say is fraud.')
print(result.message)
```

Where 'agentic' lives

You never wrote an if/else for 'first get transactions, then check fraud, then decide'. The model plans that sequence from the system prompt + tool descriptions + each result. Your job is crisp tools, a tight prompt, and hard limits — not the control flow.

Pitfall: no limits

Add a max-iterations cap and per-tool timeouts (Strands supports stop conditions / hooks). Without them a confused agent can loop on a failing API and burn tokens. Always have a graceful fallback: 'I couldn't complete this; escalating to an analyst.'

6 · Phase 3 — Expose core-banking APIs as tools (Gateway)

In-process functions don't belong in production: the agent shouldn't hold core-banking credentials, and other agents will want the same tools. Move them behind **AgentCore Gateway**, which publishes them as secured **MCP tools** and handles auth in both directions.

Recipe: REST API → MCP tool

- 1 Author/clean an **OpenAPI** spec for the core-banking transactions API; write a clear summary per operation (that text becomes the tool description the model reads). Upload the spec to S3.
- 2 Create a **Gateway**; add an **OpenAPI target** referencing the spec (use a **Lambda target** for *open_dispute_case* / *block_card*, a **Smithy target** for AWS-native services).
- 3 Configure **egress auth** (the API's OAuth/API key) via **Identity credential providers** — the secret lives in Identity, never in the agent.
- 4 Set **ingress auth** (IAM SigV4 or OAuth) so only your agents can call the Gateway.
- 5 Point the agent's MCP client at the Gateway endpoint; confirm the tools appear in discovery; test one call.

```
# Gateway target (conceptual config)
target:
  name: core-banking-transactions
  type: openapi
  schema_s3_uri: s3://bank-specs/core-banking-openapi.json
  egress_auth:
    provider: identity://providers/core-bank-oauth # secret vaulted in Identity
  ingress_auth:
    type: oauth
    discovery_url: https://login.bank.example/.well-known/openid-configuration
    allowed_audiences: [ disputes-agent ]
```

```
# In the agent: talk to Gateway over MCP instead of importing functions
from strands.tools.mcp import MCPClient
from mcp.client.streamable_http import streamablehttp_client

gateway = MCPClient(lambda: streamablehttp_client(GATEWAY_MCP_URL,
                                                  headers={'Authorization': f'Bearer {token}'}))
with gateway:
    tools = gateway.list_tools_sync() # auto-discovered from Gateway
    agent = Agent(model=model, system_prompt=SYSTEM, tools=tools)
    print(agent('Triage the $240 dispute for account 4471').message)
```

Build vs wrap

Wrap an existing REST/Lambda via Gateway when the capability already exists (most of the time). Build a custom MCP server (Guide 3) when you need richer behaviour, aggregation, or logic the API doesn't provide.

7 · Phase 4 — Memory (conversation + customer context)

The Runtime session keeps short-term context during one conversation; **AgentCore Memory** adds durable memory across turns and sessions — e.g. that this customer already disputed a similar charge last month, or their verification status.

- **Short-term**: the running dialogue for this case (handled in-session).
- **Long-term**: customer preferences, prior disputes, recurring-merchant patterns — retrieved per turn to inform decisions.
- Decide a **retention policy** and exactly which fields persist; in banking, minimise stored PII and set TTLs.
- Keep durable *case state* (status, actions taken, approvals) in DynamoDB *CaseStore* as the system of record; Memory is for context, not the audit ledger.

Pitfall

Don't let memory become a shadow copy of regulated data. Store references/IDs and minimal context; fetch sensitive details fresh via tools when needed, under current authorization.

8 · Phase 5 — Identity & authentication

Inbound (who may invoke the agent)

Front the agent with your IdP (Cognito, Okta, or Entra ID) via AgentCore **Identity inbound auth**. The console/chat obtains an OAuth bearer token; Runtime validates it before the agent runs. Analysts and customers get different scopes — a customer can ask status; only an analyst can trigger certain tools.

Outbound (how the agent calls backends)

- **User-delegated** — act as the signed-in analyst (actions attributable to a real person; ideal for audit).
- **Autonomous** — the agent's own service identity for system tasks (e.g. reading the fraud score).
- Credentials for core banking, the fraud system, and the case manager are **vaulted in Identity** and injected by Gateway at call time — they never appear in agent code, logs, or memory.

Least privilege, twice

Inbound scopes limit *who* can ask for *what*; the Gateway target's IAM/egress creds limit *what the tool itself can do*. A leaked token still can't move money because the block-card tool requires human approval (Policy, next).

9 · Phase 6 — Guardrails & Policy (the safety net)

Prompts are guidance, not security. Two deterministic layers back-stop the model:

Bedrock Guardrails (content safety)

- PII detection + redaction on inputs/outputs (mask card numbers, SSNs).
- Denied topics and word filters; block prompt-injection-style attempts to exfiltrate data.
- Apply to every model call the agent makes.

AgentCore Policy (action safety, in Cedar)

Policy intercepts **every tool call at the Gateway** and allows/denies based on which tool, what action, and under what conditions — rules the model cannot argue its way around.

```
// Cedar-style rule (illustrative)
forbid (principal, action == Action::"block_card", resource)
unless { context.human_approved == true };

permit (principal, action == Action::"open_dispute_case", resource)
when { context.amount <= 1000 && principal.role == "analyst" };
```

The safety pattern

Read-only and low-risk writes (open a case, request docs) can be autonomous.

Money-moving / customer-impacting actions (block card, provisional credit, refund) require **human-in-the-loop approval** enforced by Policy — not by the prompt. This is what makes an enterprise comfortable shipping the agent.

10 · Phase 7 — Deploy to AgentCore Runtime

Wrap the agent with the AgentCore endpoint, then configure → launch → invoke. The toolkit containerises the code, pushes to ECR, and creates the Runtime + a DEFAULT endpoint.

```
# disputes_agent.py
from bedrock_agentcore.runtime import BedrockAgentCoreApp
from strands import Agent
from strands.models import BedrockModel

app = BedrockAgentCoreApp()
model = BedrockModel(model_id='anthropic.claude-3-5-sonnet-20240620-v1:0', temperature=0.2)
agent = Agent(model=model, system_prompt=SYSTEM, tools=GATEWAY_TOOLS)

@app.entrypoint
def invoke(payload):
    return {'result': agent(payload['prompt']).message}

if __name__ == '__main__':
    app.run() # serves the runtime contract locally on :8080
```

```
# local test
python disputes_agent.py &
curl -X POST localhost:8080/invocations -H 'content-type: application/json' \
  -d '{"prompt":"Triage $240 dispute for account 4471"}'

# deploy: configure -> launch -> invoke
agentcore configure -e disputes_agent.py
agentcore launch # builds container, pushes to ECR, creates Runtime
agentcore invoke '{"prompt":"Triage $240 dispute for account 4471"}'
```

- Each config change makes an immutable **version**; create distinct **dev/staging/prod endpoints**.
- Sessions are isolated microVMs (up to 8h), so long investigations and multi-turn dialogues are safe and private.
- Roll out by pointing the prod endpoint at a new version; roll back by repointing — zero downtime.

11 · Phase 8 — Observability & Evaluations

- **Observability** — OTEL traces show each step: which tools ran, inputs/outputs, latency, the Policy decision, and where it failed. Debugging a wrong action = reading the trace, not the logs.
- **Alarms** — error rate, p95 latency, tokens + tool-calls per case (cost), and business KPIs like auto-resolve rate and escalation rate.
- **Evaluations** — an eval set of representative cases (fraud / duplicate / not-received, edge cases, attempted manipulation) with expected behaviour; score sessions automatically and gate deploys on no-regression.
- **Canary** — release a new version to a small % of traffic via a separate endpoint, watch metrics, then promote.

Build the eval set from real cases

Seed it with resolved historical cases (de-identified). It becomes your regression suite and your evidence to risk/compliance that the agent behaves within policy before and after each change.

12 · Security & compliance deep dive

Control	How it's applied here
Least privilege	Per-tool execution roles; scoped Gateway egress; inbound scopes per role
Secrets	Backend creds vaulted in Identity / Secrets Manager; none in code/logs
Encryption	KMS CMK on S3, DynamoDB, logs; TLS in transit; VPC endpoints keep traffic private
PII / PHI	Guardrails redact PII; memory stores minimal context; data-class tags drive access
Action limits	Cedar Policy + human approval on money-moving/customer-impacting actions

Control	How it's applied here
Auditability	CloudTrail + case ledger in DynamoDB + OTEL traces = full replay of any decision
Segregation	Separate dev/staging/prod accounts; prod data never in lower envs

Regulatory note (not legal advice)

Banking touches PCI-DSS (card data), SOX (controls), and consumer-protection dispute timeframes. Treat the agent as a regulated system: documented controls, change management, human oversight on adverse actions, and a complete audit trail. Get your risk/compliance and legal teams in early — flag anything ambiguous for human review.

13 · Cost model & estimate

Cost $\approx$ per-case (model tokens across the loop) + (tool/Gateway calls) + (Runtime session time) + (Memory + storage) + (Knowledge Base queries). Control it with the levers from Guide 1:

- Cap loop iterations; keep tool outputs small and structured (biggest token lever).
- Use a cheaper model for simple sub-steps (classification, extraction) and the strong model only for judgement.
- Cache stable lookups; avoid re-fetching the same transactions each turn.
- Tag everything so cost is attributable per team and per case type; alarm on cost-per-case drift.

Make the ROI explicit

Put cost-per-case next to analyst-minutes-saved in one dashboard. If auto-resolve is 45% at a fraction of an analyst's cost per case, the business case writes itself — and you can see immediately if a change erodes it.

14 · End-to-end data flow (one dispute, start to finish)

- 1 Customer opens chat: "I didn't make a \$240 charge." The app attaches their OAuth token + a stable session id.
- 2 Runtime validates the token (Identity inbound); the Triage Agent starts in an isolated session.
- 3 Agent loads prior context from Memory (e.g. verification status, recent disputes).
- 4 Agent calls *get_transactions* via Gateway (egress auth to core banking) and finds the \$240 charge.
- 5 Agent calls *get_fraud_signals*; the fraud model returns a high anomaly score + new-merchant signal.
- 6 Agent queries the Knowledge Base for the fraud-dispute policy; it cites the provisional-credit rule and timeframe.

- 7 Decision: open a dispute case (low-risk) — Policy permits (amount under limit, role ok) — via *open_dispute_case* with an idempotency key. Case written to DynamoDB.
- 8 Blocking the card would be high-risk: Policy **denies** autonomous action and the agent routes an approval task to an analyst with the full recommendation + cited policy.
- 9 Guardrails redact the card number in the reply; the agent tells the customer a case is open, the provisional-credit timeline, and that a verification step is pending.
- 10 Every step streamed to Observability; the case ledger + CloudTrail give a complete, replayable audit record.

Notice the shape

request → inbound auth → agent loop (Bedrock) → tools via Gateway (egress auth + Policy) → Memory + Knowledge Base → human gate on the risky action → guarded response, fully traced. Every enterprise agent you build reuses this exact skeleton.

15 · Go-live runbook, failure modes & rollback

Go-live checklist

- Iteration cap, per-tool timeouts, and graceful fallback verified.
- Inbound auth enforced; scopes correct for analyst vs customer.
- Every tool runs least-privilege; backend creds in Identity; nothing in logs.
- Guardrails on; Cedar Policy gates all money-moving/customer-impacting actions + human approval wired.
- Prod endpoint separate from dev/staging; versioning + tested rollback.
- Observability + alarms live (error, latency, cost-per-case, auto-resolve, escalation).
- Eval set passes; canary plan ready; risk/compliance sign-off obtained.

Failure modes & mitigations

If this happens	Mitigation
Core-banking API down	Tool timeout + retry; agent degrades to 'escalate to analyst'
Agent loops / stalls	Max-iterations cap returns a safe fallback; alarm fires
Wrong/risky action proposed	Policy denies autonomous execution; human approval catches it
Cost spikes	Cost-per-case alarm; inspect trace for runaway tool calls; trim context
Bad new version	Repoint prod endpoint to previous version (instant rollback)
PII leak risk	Guardrail redaction + minimal memory; trace audit to confirm

Where this leads

You now have a governed, observable enterprise agent end to end. **Guide 3** zooms into the tool layer: how to build that capability set as a reusable, governed **MCP platform** the whole bank — and your coding assistants — can share.

Sources: AWS Bedrock AgentCore developer guide (Runtime, Gateway, Identity, Memory, Observability, Policy); Strands Agents docs (deploy to AgentCore Runtime); AWS ML blog. [docs.aws.amazon.com / strandsagents.com](https://docs.aws.amazon.com/strandsagents.com). Verified May 2026.